Propuestas de Proyectos Finales

Algoritmos y Programación — Python con Interfaz Gráfica

Prof. D.Sc. Aboud Barsekh-Onji

2026 - enero-mayo

Índice

Lineamientos Generales del Proyecto Final **3**
Criterios de Evaluación . . . 3
Entregables . . . 3

Proyecto 1 — Control de Almacén **4**
Descripción General . . . 4
Funcionalidades Requeridas . . . 4
Estructuras de Python que Debe Usar . . . 4
Restricciones . . . 4

Proyecto 2 — Sistema de Calificaciones Estudiantiles **5**
Descripción General . . . 5
Funcionalidades Requeridas . . . 5
Estructuras de Python que Debe Usar . . . 5
Restricciones . . . 5

Proyecto 3 — Juego de Trivia / Preguntas y Respuestas **6**
Descripción General . . . 6
Funcionalidades Requeridas . . . 6
Estructuras de Python que Debe Usar . . . 6
Restricciones . . . 6

Proyecto 4 — Simulador de Cajero Automático (ATM) **7**
Descripción General . . . 7
Funcionalidades Requeridas . . . 7
Estructuras de Python que Debe Usar . . . 7
Restricciones . . . 7

Proyecto 5 — Agenda Personal / Gestor de Tareas **8**
Descripción General . . . 8
Funcionalidades Requeridas . . . 8
Estructuras de Python que Debe Usar . . . 8
Restricciones . . . 8

Proyecto 6 — Generador de Contraseñas Seguras **9**
Descripción General 9
Funcionalidades Requeridas 9
Estructuras de Python que Debe Usar 9
Restricciones 9

Proyecto 7 — Simulador de Lotería / Sorteos **10**
Descripción General 10
Funcionalidades Requeridas 10
Estructuras de Python que Debe Usar 10
Restricciones 10

Proyecto 8 — Calculadora Científica con Historial **11**
Descripción General 11
Funcionalidades Requeridas 11
Estructuras de Python que Debe Usar 11
Restricciones 11

Proyecto 9 — Sistema de Registro para Torneo Deportivo **12**
Descripción General 12
Funcionalidades Requeridas 12
Estructuras de Python que Debe Usar 12
Restricciones 12

Proyecto 10 — Simulador de Examen / Quiz con Retroalimentación **13**
Descripción General 13
Funcionalidades Requeridas 13
Estructuras de Python que Debe Usar 13
Restricciones 13

Resumen de Proyectos **14**

Lineamientos Generales del Proyecto Final

El proyecto final representa el **25% de la calificación total** del curso. Cada alumno desarrollará de manera **individual** un software funcional con **interfaz gráfica en Python** (usando `tkinter`), que integre los temas vistos durante el semestre: estructuras de control (`if`, `match`, `for`, `while`), ciclos anidados, números aleatorios (`random.randint`, `random.choice`), funciones, listas, matrices con noción básica de programación orientada a objetos (POO).

Los proyectos serán **asignados mediante sorteo**. Cada alumno recibirá uno de los diez proyectos descritos en este documento.

Criterios de Evaluación

Criterio	Ponderación
Funcionalidad completa del programa	40%
Uso correcto de estructuras vistas en clase	25%
Interfaz gráfica con `tkinter`	20%
Organización del código (funciones / clases)	10%
Presentación y demostración	5%

Entregables

1. Archivo `.py` con el código fuente completo.
2. Documento PDF (máximo 3 páginas) con: descripción del programa, diagrama de flujo principal y capturas de pantalla.
3. Demostración en vivo frente al profesor (5 minutos).

Proyecto 1 — Control de Almacén

Descripción General

Sistema de gestión de inventario para un pequeño almacén o tienda. El programa permite registrar productos, consultar existencias, registrar entradas y salidas de mercancía, y emitir un reporte final del estado del inventario.

Funcionalidades Requeridas

- **Alta de producto:** formulario para ingresar nombre, categoría, cantidad inicial y precio unitario. Los datos se almacenan en listas paralelas o una lista de listas.
- **Búsqueda de producto:** el usuario escribe el nombre o código del producto; el programa recorre la lista con un ciclo `for` y muestra la información encontrada.
- **Registro de entrada / salida:** actualiza la cantidad disponible sumando o restando unidades. Debe validar con `if` que no se registre una salida mayor al stock existente.
- **Reporte de inventario:** muestra todos los productos en una tabla dentro de la ventana, ordenados por nombre (usando `.sort()`). Calcula el valor total del inventario (precio $\times$ cantidad para cada producto).
- **Alerta de stock bajo:** al consultar el reporte, resalta en color rojo los productos cuya cantidad sea menor a un umbral definido por el usuario.

Estructuras de Python que Debe Usar

`for` (recorrido de listas), `if / elif / else` (validaciones), funciones (`def`), listas, métodos de lista (`.append()`, `.sort()`), `tkinter` (ventana principal, `Label`, `Entry`, `Button`, `Listbox` o `Text`).

Restricciones

- No se permite el uso de bases de datos externas ni archivos CSV (todo en memoria durante la ejecución).
- El programa debe manejar al menos **5 productos simultáneos** de demostración.

Proyecto 2 — Sistema de Calificaciones Estudiantiles

Descripción General

Aplicación para que un profesor registre y analice las calificaciones de sus alumnos en distintas evaluaciones. El sistema calcula promedios, identifica al alumno con mayor y menor calificación, y genera estadísticas básicas del grupo.

Funcionalidades Requeridas

- **Registro de alumnos:** permite agregar alumnos con nombre y matrícula a una lista.
- **Captura de calificaciones:** para cada alumno se ingresan tres calificaciones (parcial 1, parcial 2, final). Se almacenan en listas anidadas o listas paralelas.
- **Cálculo de promedio individual:** al seleccionar un alumno, muestra sus tres calificaciones y el promedio ponderado. Usa `if` para mostrar si el alumno aprobó o reprobó (umbral: 6.0).
- **Estadísticas del grupo:** calcula y muestra promedio grupal, calificación máxima, calificación mínima, y cuenta cuántos alumnos aprobaron y cuántos reprobaron. Usa ciclos `for` para recorrer todas las calificaciones.
- **Asignación aleatoria de equipo:** botón que usa `random.choice` para asignar aleatoriamente a cada alumno a uno de tres equipos de trabajo (Equipo A, B o C).
- **Tabla de resultados:** muestra la lista completa de alumnos con sus promedios y status (Aprobado / Reprobado) en un widget de texto o tabla.

Estructuras de Python que Debe Usar

`for`, `while` (para validar entradas), `if / elif / else`, funciones, listas anidadas, `random.choice`, `tkinter`.

Restricciones

- El grupo debe poder tener entre **5 y 30 alumnos**.
- Los promedios deben mostrarse con **dos decimales**.

Proyecto 3 — Juego de Trivia / Preguntas y Respuestas

Descripción General

Juego interactivo de preguntas y respuestas de opción múltiple sobre un tema a elección del alumno (ciencia, cultura general, deportes, etc.). El programa presenta preguntas en orden aleatorio, lleva la puntuación y al final muestra un resultado con categoría de desempeño.

Funcionalidades Requeridas

- **Banco de preguntas:** el programa contiene al menos **15 preguntas** almacenadas en listas. Cada pregunta tiene: texto de la pregunta, cuatro opciones (lista), índice de la respuesta correcta y una explicación breve.
- **Selección aleatoria:** al iniciar, el programa selecciona **10 preguntas** de manera aleatoria sin repetición usando `random` (puede usar `random.randint` con control de repetición o `random.sample`).
- **Presentación de pregunta:** muestra la pregunta y cuatro botones de opción. Al dar clic, indica si la respuesta fue correcta o incorrecta y muestra la explicación.
- **Temporizador por pregunta:** el jugador tiene 20 segundos para responder cada pregunta. Usa el método `.after()` de `tkinter` para implementar la cuenta regresiva. Si el tiempo se agota, se cuenta como respuesta incorrecta.
- **Puntuación acumulada:** lleva el conteo de respuestas correctas. Al terminar las 10 preguntas, muestra la puntuación final y una categoría:
  - 9–10 correctas: *Experto*
  - 7–8: *Avanzado*
  - 5–6: *Intermedio*
  - Menos de 5: *Principiante*
- **Reinicio de juego:** botón para volver a jugar con nuevas preguntas aleatorias.

Estructuras de Python que Debe Usar

`for`, `if / elif / else` (categorías de desempeño, validación de respuestas), `match` (para clasificar categoría de puntaje), `random.randint` o `random.sample`, funciones, listas, `tkinter` (incluyendo `.after()` para el temporizador).

Restricciones

- Las preguntas **no deben repetirse** en una misma partida.
- El tema de las preguntas debe quedar claro en la pantalla de bienvenida.

Proyecto 4 — Simulador de Cajero Automático (ATM)

Descripción General

Simulación de un cajero automático que permite a un usuario iniciar sesión con un NIP, consultar saldo, realizar depósitos y retiros, y consultar un historial de movimientos. El sistema maneja múltiples cuentas de usuario.

Funcionalidades Requeridas

- **Pantalla de inicio de sesión:** el usuario ingresa un número de cuenta (4 dígitos) y un NIP (4 dígitos). El programa valida con `if` que el par cuenta-NIP sea correcto. Permite **3 intentos** antes de bloquear el acceso (ciclo `while` con contador).
- **Menú principal:** tras autenticarse, muestra opciones: Consultar Saldo, Depositar, Retirar, Ver Movimientos, Salir.
- **Consulta de saldo:** muestra el saldo actual de la cuenta autenticada.
- **Depósito:** el usuario ingresa un monto; se valida que sea positivo y múltiplo de 100 (usando `%`). Actualiza el saldo y registra el movimiento con descripción y monto.
- **Retiro:** valida que el monto sea múltiplo de 100, positivo, y que haya saldo suficiente. Los montos disponibles en el cajero son: $100, $200, $500. Usa `random.choice` para simular la "selección de billetes" disponibles y muestra qué billetes entrega.
- **Historial de movimientos:** muestra la lista de todos los movimientos de la sesión actual (tipo, monto, saldo resultante) recorriendo la lista con `for`.
- **Datos precargados:** el sistema debe tener al menos **3 cuentas** con datos iniciales (número, NIP, nombre del titular, saldo).

Estructuras de Python que Debe Usar

`while` (sesión de intentos, menú principal), `for` (historial), `if / elif / else`, `match` (opciones del menú), `random.choice`, funciones, listas, `tkinter`.

Restricciones

- No mostrar el NIP en pantalla en ningún momento (usar `show="*"` en el `Entry`).
- El saldo nunca puede ser negativo.

Proyecto 5 — Agenda Personal / Gestor de Tareas

Descripción General

Aplicación de productividad personal para registrar tareas pendientes, asignarles prioridad y fecha de vencimiento, marcarlas como completadas y filtrar las tareas por estado o prioridad.

Funcionalidades Requeridas

- **Agregar tarea:** formulario con campos: descripción, prioridad (Alta / Media / Baja) y fecha de vencimiento (día/mes/año como texto). Los datos se guardan en una lista de diccionarios o lista de listas.
- **Visualización de tareas:** muestra todas las tareas en una lista o tabla con su descripción, prioridad, fecha y estado (Pendiente / Completada). Usa `for` para recorrer y mostrar.
- **Marcar como completada:** al seleccionar una tarea y presionar un botón, cambia su estado a *Completada* y la muestra tachada o en color gris.
- **Filtros:** botones para mostrar: Todas las tareas, Solo pendientes, Solo completadas, Solo de alta prioridad. Usa `if` dentro de un ciclo `for` para filtrar.
- **Ordenar tareas:** botón para ordenar la lista por prioridad (Alta primero) o por fecha. Usa la función `.sort()` con una llave de ordenamiento (`key=`).
- **Tarea aleatoria del día:** botón que usa `random.choice` para seleccionar y resaltar una tarea pendiente aleatoria como *"Tarea del día"*.
- **Estadísticas:** muestra cuántas tareas hay en total, cuántas están completas y el porcentaje de avance.

Estructuras de Python que Debe Usar

`for`, `if / elif / else`, `match` (para prioridades), funciones, listas, `random.choice`, `tkinter`.

Restricciones

- El programa debe soportar al menos **10 tareas simultáneas** en demostración.
- La interfaz debe ser limpia y fácil de usar.

Proyecto 6 — Generador de Contraseñas Seguras

Descripción General

Herramienta de seguridad informática que genera contraseñas aleatorias según criterios configurables por el usuario, evalúa la fortaleza de contraseñas existentes, y mantiene un historial de contraseñas generadas en la sesión.

Funcionalidades Requeridas

- **Panel de configuración:** el usuario elige mediante checkboxes y sliders: longitud de la contraseña (8–32 caracteres), incluir mayúsculas (`A-Z`), minúsculas (`a-z`), dígitos (`0-9`) y caracteres especiales (`!@#\$%^&*`). Valida con `if` que al menos un tipo de carácter esté seleccionado.
- **Generación de contraseña:** construye una lista con todos los caracteres permitidos según la configuración, luego usa `random.choice` en un ciclo `for` para generar la contraseña carácter a carácter. Asegura con `if` que al menos un carácter de cada tipo seleccionado esté presente.
- **Medidor de fortaleza:** evalúa la contraseña y la clasifica en:
  - *Muy débil*: menos de 8 caracteres o solo un tipo.
  - *Débil*: 8–11 caracteres con dos tipos.
  - *Moderada*: 12–15 caracteres con tres tipos.
  - *Fuerte*: 16+ caracteres con los cuatro tipos. Usa `match` o `if / elif` para la clasificación. Muestra una barra de color (rojo, naranja, amarillo, verde) correspondiente a la fortaleza.
- **Múltiples contraseñas:** permite generar de 1 a 10 contraseñas en una sola operación usando un ciclo `for`. Las muestra en una lista seleccionable.
- **Historial de sesión:** lista con todas las contraseñas generadas durante la sesión. Permite copiar al portapapeles cualquiera de ellas.
- **Evaluador externo:** campo donde el usuario escribe cualquier texto y el programa evalúa y califica su fortaleza como contraseña.

Estructuras de Python que Debe Usar

`for`, `while` (garantizar al menos un carácter de cada tipo), `if / elif / else`, `match`, `random.choice`, `random.randint`, funciones, listas, `tkinter`.

Restricciones

- Las contraseñas generadas **no deben repetirse** en el historial de la sesión.
- El botón de copiar debe usar `clipboard_clear()` y `clipboard_append()` de `tkinter`.

Proyecto 7 — Simulador de Lotería / Sorteos

Descripción General

Aplicación de entretenimiento que simula distintos tipos de sorteos y juegos de azar: lotería numérica, ruleta simplificada y sorteo de nombres. El programa lleva estadísticas de los sorteos realizados.

Funcionalidades Requeridas

- **Lotería numérica:** el usuario elige 6 números del 1 al 49. El programa genera 6 números ganadores aleatorios sin repetición usando `random.randint` y verificación con `if` dentro de un ciclo `while`. Compara los números del usuario con los ganadores usando `for` y muestra cuántos aciertos hubo con el premio correspondiente:
  - 6 aciertos: *Primer premio*
  - 5 aciertos: *Segundo premio*
  - 4 aciertos: *Tercer premio*
  - Menos de 4: *Sin premio* Usa `match` para mostrar el resultado.
- **Ruleta simplificada:** el usuario elige entre *Rojo*, *Negro* o *Verde*. El programa simula el giro con `random.choice` sobre una lista de 37 opciones (18 rojas, 18 negras, 1 verde). Muestra el resultado con animación de texto (varios `.after()` que simulan el giro).
- **Sorteo de nombres:** el usuario ingresa una lista de nombres (mínimo 3). El programa los desordena con `random` y realiza el sorteo uno por uno mostrando el ganador con animación.
- **Estadísticas de sesión:** lleva conteo de: cuántas veces se ha jugado cada modalidad, historial de resultados de lotería (aciertos obtenidos), y frecuencia de aparición de cada color en la ruleta. Muestra los datos con ciclo `for`.

Estructuras de Python que Debe Usar

`for`, `while` (números sin repetición), `if / elif / else`, `match`, `random.randint`, `random.choice`, funciones, listas, `tkinter` (incluyendo `.after()` para animaciones).

Restricciones

- Los 6 números ganadores de la lotería **no deben repetirse**.
- La animación de la ruleta debe mostrar al menos 5 “giros” antes del resultado final.

Proyecto 8 — Calculadora Científica con Historial

Descripción General

Calculadora avanzada con interfaz gráfica que, además de las operaciones aritméticas básicas, incluye funciones matemáticas del módulo `math`, conversión de unidades y un historial completo de operaciones realizadas en la sesión.

Funcionalidades Requeridas

- **Calculadora básica:** operaciones de suma, resta, multiplicación, división, potencia y módulo. Valida con `if` la división entre cero y muestra un mensaje de error adecuado.
- **Funciones científicas:** botones para: raíz cuadrada (`math.sqrt`), valor absoluto (`abs`), logaritmo natural (`math.log`), logaritmo base 10 (`math.log10`), seno, coseno y tangente (en grados, convirtiendo con `math.radians`), factorial (`math.factorial`). Valida con `if` que los argumentos sean válidos (ej. raíz de número negativo).
- **Historial de operaciones:** cada operación realizada se guarda en una lista con la expresión y el resultado. Se muestra en un panel lateral. El usuario puede hacer clic en una entrada del historial para reutilizar ese resultado.
- **Conversión de unidades:** panel con selector de categoría (`match`): Longitud (metros, km, millas, pies), Temperatura (°C, °F, K) o Peso (kg, lb, oz). Usa funciones para cada conversión.
- **Modo aleatorio / "Desafío":** botón que genera una operación matemática aleatoria usando `random.randint` y `random.choice` (elige operador y operandos al azar), y pide al usuario que la resuelva. Valida la respuesta y lleva un marcador de aciertos.
- **Borrar historial:** botón para limpiar la lista de historial con confirmación.

Estructuras de Python que Debe Usar

`for` (recorrido del historial), `if / elif / else` (validaciones), `match` (categorías de conversión, operadores), módulo `math`, `random.randint`, `random.choice`, funciones, listas, `tkinter`.

Restricciones

- El historial debe mostrar al menos las últimas **20 operaciones**.
- Los resultados deben mostrarse con hasta **6 cifras decimales significativas**.

Proyecto 9 — Sistema de Registro para Torneo Deportivo

Descripción General

Aplicación para gestionar un torneo deportivo (fútbol, basquetbol u otro deporte a elección). Permite registrar equipos, generar el fixture de partidos de manera aleatoria, registrar resultados y mostrar la tabla de posiciones actualizada.

Funcionalidades Requeridas

- **Registro de equipos:** permite agregar equipos con nombre y color/ciudad. Almacena los datos en listas. Valida con `if` que no se repitan nombres y que el número de equipos sea par (mínimo 4, máximo 8).
- **Generación de fixture (ida):** una vez registrados los equipos, genera los enfrentamientos de todos contra todos (round-robin). Usa ciclos `for` anidados para generar los pares de equipos, y `random.choice` para decidir quién juega como local. Muestra el fixture completo.
- **Registro de resultados:** al seleccionar un partido del fixture, el usuario ingresa los goles de cada equipo. El programa actualiza automáticamente la tabla de posiciones (3 puntos por victoria, 1 por empate, 0 por derrota) usando `if / elif`.
- **Tabla de posiciones:** muestra todos los equipos ordenados por puntos (`.sort()`), con columnas: Posición, Equipo, PJ, PG, PE, PP, GF, GC, Diferencia, Puntos.
- **Estadísticas del torneo:** calcula con ciclos `for`: máximo goleador por equipo, partido con más goles, equipo con mejor defensa (menos goles en contra).
- **Sorteo de campeón ficticio:** si el torneo termina en empate de puntos entre dos equipos, un botón usa `random.choice` para simular un "desempate por penales" y declarar al campeón.

Estructuras de Python que Debe Usar

`for` (fixture, tabla), `for` anidado (generación de enfrentamientos), `if / elif / else` (puntos, validaciones), `match` (resultado del partido), `random.choice`, funciones, listas, `tkinter`.

Restricciones

- El número de equipos debe ser **par** (validado al intentar generar el fixture).
- La tabla debe actualizarse **en tiempo real** cada vez que se registra un resultado.

Proyecto 10 — Simulador de Examen / Quiz con Retroalimentación

Descripción General

Sistema para que un profesor cree un banco de reactivos y genere exámenes aleatorios para sus alumnos. El programa simula la aplicación de un examen, califica automáticamente, y presenta una retroalimentación detallada al finalizar.

Funcionalidades Requeridas

- **Modo profesor — Banco de reactivos:** formulario para agregar preguntas al banco. Cada pregunta incluye: texto, cuatro opciones, respuesta correcta (A/B/C/D), tema al que pertenece y nivel de dificultad (Fácil / Medio / Difícil). Se almacenan en una lista de listas. El banco debe tener al menos **20 preguntas precargadas** sobre temas de programación en Python.
- **Modo alumno — Generación de examen:** el alumno ingresa su nombre y elige cuántas preguntas quiere (5, 10 ó 15). El programa selecciona aleatoriamente con `random.randint` las preguntas del banco garantizando que no se repitan. Usa `match` para determinar la cantidad.
- **Aplicación del examen:** presenta una pregunta a la vez con sus cuatro opciones como botones de radio. Lleva un contador de pregunta actual / total. El alumno puede navegar a la pregunta anterior o siguiente sin perder sus respuestas (almacenadas en una lista).
- **Calificación automática:** al enviar el examen, recorre con `for` todas las respuestas, compara con las correctas y calcula la calificación en escala 0–10.
- **Retroalimentación detallada:** muestra pregunta por pregunta: el texto de la pregunta, la respuesta del alumno (en verde si correcta, rojo si incorrecta) y la respuesta correcta. Usa `for` para recorrer y `if` para colorear.
- **Estadísticas del examen:** muestra: calificación final, número de correctas e incorrectas, porcentaje de aciertos por tema (usa `for` y `if` para agrupar), y una recomendación basada en el puntaje.
- **Historial de exámenes:** lista con todos los exámenes aplicados en la sesión (nombre del alumno, fecha-hora como texto, calificación).

Estructuras de Python que Debe Usar

`for` (calificación, retroalimentación, estadísticas por tema), `for` anidado (comparación por tema), `if / elif / else`, `match` (número de preguntas, categoría de calificación), `while` (validación de entradas), `random.randint`, `random.choice`, funciones, listas, `tkinter`.

Restricciones

- Las preguntas del examen **no deben repetirse**.
- El programa debe distinguir claramente el **Modo Profesor** del **Modo Alumno** (puede ser con una contraseña simple para el modo profesor).

Resumen de Proyectos

No.	Proyecto	Tema Principal	Complejidad
1	Control de Almacén	Listas, búsqueda, ordenamiento	Media
2	Calificaciones Estudiantiles	Estadísticas, listas anidadas	Media
3	Juego de Trivia	Aleatoriedad, temporizador	Media-Alta
4	Simulador de ATM	Validación, sesiones, menú	Media-Alta
5	Agenda / Gestor de Tareas	Filtros, ordenamiento, POO	Media
6	Generador de Contraseñas	Aleatoriedad, seguridad	Media
7	Simulador de Lotería	Aleatoriedad, animación	Media-Alta
8	Calculadora Científica	Módulo `math`, historial	Media
9	Torneo Deportivo	Ciclos anidados, tabla dinámica	Alta
10	Simulador de Examen	Banco de reactivos, estadísticas	Alta

Facultad de Ingeniería — Universidad Anáhuac México
Prof. D.Sc. Aboud Barsekh-Onji | `aboud.barsekh@anahuac.mx`